CLAIMGUARD

System Architecture

Insurance Claims Processing with Fraud-Pattern Detection

Document	CLAIMGUARD_ARCH
Version	2.0 — Consolidated team baseline
Date	23 October 2026
Team member 1	Hrushikesh Karnam — PES1UG24CS668
Team member 2	Nishant Hegde — PES1UG24CS681
Project context	Database Management Systems — Level 3 Experiential Learning Project
Status	Consolidated draft for team review and implementation traceability

Prepared for the ClaimGuard project team and course evaluator

Document control and baseline

Item	Description
Requirements source	CLAIMGUARD_SRS v2.0, consolidated from the team SRS v1.1 and earlier requirements.
Architecture style	Modular monolith with one stateless embedding helper service.
Runtime	Three local processes: Next.js server, Python embedding service, PostgreSQL 16 + pgvector.
Data access	node-postgres (pg), parameterized SQL, numbered SQL migrations as schema source of truth.
Model	sentence-transformers/all-MiniLM-L6-v2; normalized 384-dimensional embeddings.
Deployment	Local development/demo; no Docker or message broker required.
Status	Design baseline. A specified object/endpoint is not considered implemented until verified in the repository and tests.

1. Architecture overview

1.1 Architectural style

ClaimGuard is a modular monolith with a separate embedding helper service. The web UI and backend Route Handlers live in one Next.js deployable, separated internally into claim, screening, review, admin and reporting modules. Python is used only for embedding inference. PostgreSQL is the system of record for structured data, vectors, screening outcomes, decisions and audit events.

- Browser traffic terminates at the Next.js server; the browser never connects directly to PostgreSQL or the embedding service.
- Next.js is the only application component that accesses PostgreSQL and invokes Python inference.
- Critical data invariants are enforced in PostgreSQL, not only in UI or application code.
- Screening is asynchronous in state, not infrastructure: the claim is committed as PENDING before screening; no durable message broker is introduced.
- Similarity and risk indicators are advisory only; the final claim decision belongs to an authorized adjuster.

1.2 System context

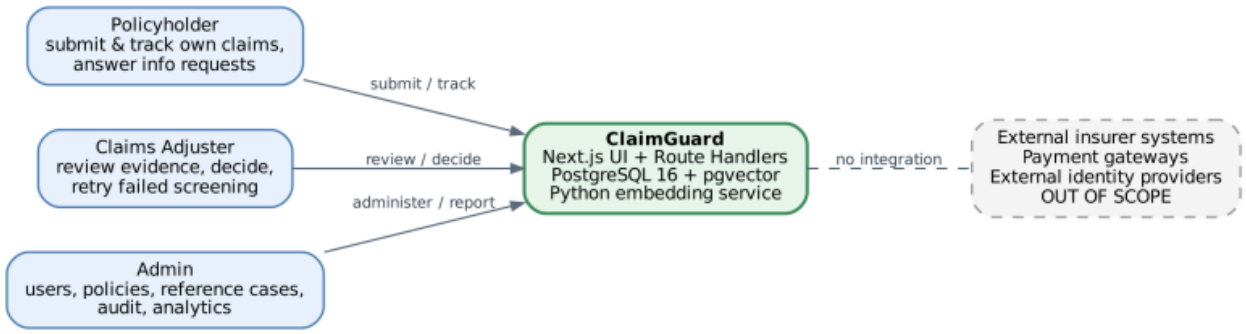


Figure 1. System context. External insurer, payment and identity-provider integrations are out of scope.

Figure 1. System context. External insurer, payment and identity-provider integrations are out of scope.

2.2 Component architecture

The browser talks only to the Next.js server. Only the server calls the embedding service and the database.

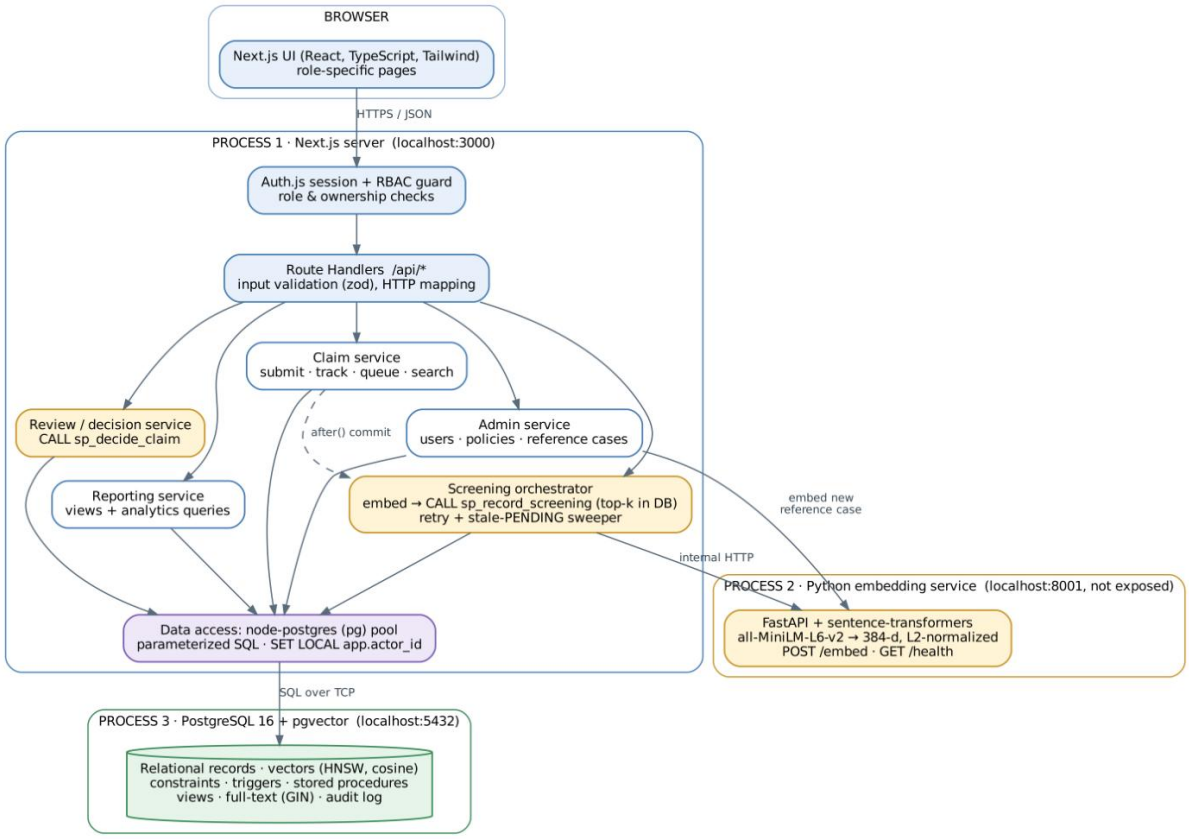


Figure 2. Component architecture and the three local processes.

Component	Responsibilities	Technology / boundary
Frontend	Role-specific claim form, claim tracking, notifications, adjuster queue/detail, admin screens and reports.	Next.js, React, TypeScript, Tailwind CSS.
API layer	Request validation, session resolution, role/ownership checks, HTTP error mapping and service invocation.	Next.js Route Handlers; Zod or equivalent validation.
Authentication/RBAC	Credential verification, password-hash checking, session creation, role and ownership authorization.	Auth.js credentials flow and bcrypt-compatible hashing, as implemented.
Claim service	Submission, tracking, queue, filters and information responses.	TypeScript module; parameterized SQL.
Screening orchestrator	Calls embedding service, invokes screening procedures, records retries and recovers stale work.	TypeScript module; explicit state machine.
Embedding service	Returns normalized 384-D vectors and model identity/version; no database access.	Python, FastAPI, Sentence Transformers; local HTTP service.
Review/decision service	Starts review and submits approve/reject/request-information action through guarded transaction.	TypeScript module plus sp_decide_claim or verified equivalent.
Admin service	User/role management, policy issuance, reference cases, risk-rule versions and audit inspection.	TypeScript module; audited database operations.
Reporting service	Operational summaries, analytical queries and ranked keyword search.	SQL views, aggregates, subqueries and full-text index.
PostgreSQL	Stores relational entities, vectors, attempts,	PostgreSQL 16 + pgvector; constraints, triggers,

Component	Responsibilities	Technology / boundary
	matches, risk rules, reviews, decisions, notifications and audit events.	procedures, views and indexes.

1.4 Technology decisions

Decision	Rationale and limitation
PostgreSQL 16	Policies, claims, ownership, reviews and decisions require relational integrity, transactions and guarded state changes.
pgvector	Stores embeddings with relational data and enables semantic nearest-neighbour retrieval without a separate vector database.
all-MiniLM-L6-v2	Compact model for local CPU inference; outputs 384 dimensions. Similarity quality is domain- and dataset-dependent.
Cosine distance + HNSW	Suitable for normalized embeddings; HNSW is approximate, so recall and performance must be measured on actual data.
node-postgres + SQL migrations	Keeps database logic explicit, supports procedures/triggers/vector types and avoids a parallel ORM schema.
No Docker or message broker	Matches the local project baseline. A process-based sweeper is not equivalent to durable queue infrastructure.

2. Runtime and trust boundaries

Figure 3. Local runtime: each developer runs three processes; secrets remain in a git-ignored environment file.

- Process 1 — Next.js server/UI/API, conventionally on localhost:3000.
- Process 2 — Python embedding service, conventionally bound to 127.0.0.1:8001 and not exposed to the browser.
- Process 3 — PostgreSQL 16 with pgvector, conventionally on localhost:5432 or the configured database host.
- Store credentials in .env.local or equivalent ignored configuration; commit only a safe .env.example.
- Use parameterized SQL for all dynamic values. Never concatenate user input into SQL.
- Set audit actor identity with trusted server-controlled transaction context (for example, set_config('app.actor_id', ..., true)); do not trust a browser-supplied actor ID.
- Use least-privilege database credentials where feasible. This is a student prototype, not a production security certification.

3. Data flow

3.1 Claim submission and screening

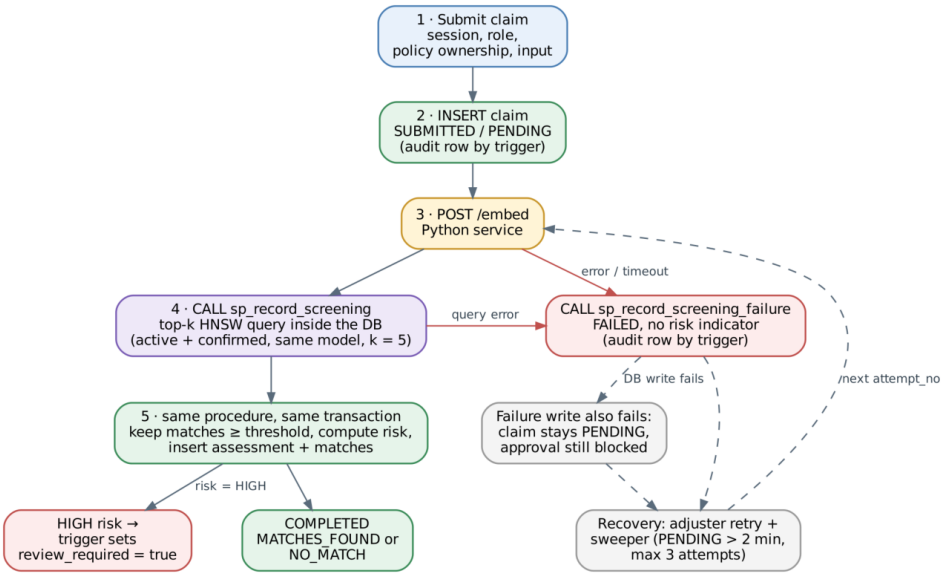


Figure 3. Screening flow. A failed or unfinished screening never means low risk and never permits approval.

1. Validate session, policy ownership, policy active status, incident date, amount and description.
2. Within a database transaction, insert the claim as SUBMITTED with screening PENDING. The claim-submission audit event is created by the configured database trigger.
3. After commit, the Next.js orchestrator sends the original description to the Python embedding service.
4. The embedding service returns a normalized 384-dimensional vector plus model name/version. It has no database credentials.
5. The orchestrator invokes the screening procedure with the claim, model identity and vector. The procedure locks the claim/attempt state, loads the active risk rule, retrieves top-k eligible active/confirmed references of the same model, computes scores and persists the assessment and matches.
6. On a completed attempt, persist COMPLETED with MATCHES_FOUND or NO_MATCH. A NO_MATCH result is a successful screening outcome, not a failure.
7. If risk is HIGH, a database trigger sets review_required. This is a request for manual review, not an automated accusation or denial.
8. If inference or persistence fails, record FAILED using the failure procedure where possible. Do not downgrade to LOW risk or allow approval.

3.2 Screening state and retry

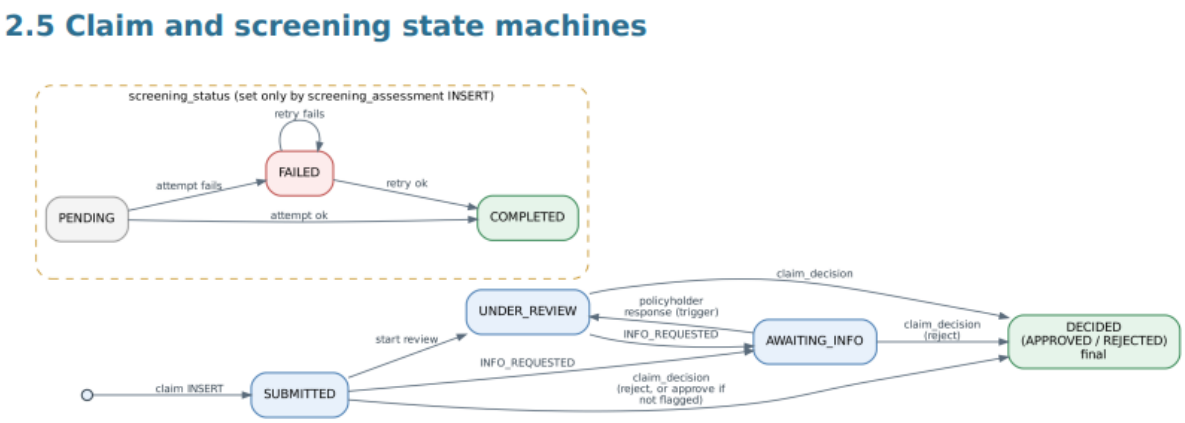


Figure 5. claim_status transitions (enforced by fn_claim_state_guard) and screening_status transitions.

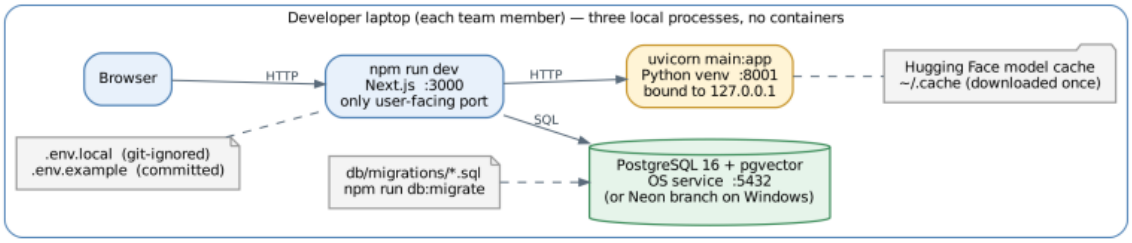


Figure 6. Each developer runs three processes. Secrets live in a git-ignored .env.local file.

Figure 5. Claim-status and screening-status state machines from the team architecture baseline.

- Attempt numbers are allocated under a row lock or equivalent transaction-safe mechanism and protected by UNIQUE(claim_id, attempt_no).
- An adjuster can retry a FAILED attempt. A lightweight Next.js sweeper can retry stale PENDING work and configured FAILED attempts.
- Use a bounded retry policy; the teammate baseline specifies a sweep every 60 seconds, considers PENDING older than 2 minutes stale, and caps automatic retries at 3. Align code and tests if these values are adopted.
- The sweeper is best-effort local recovery, not a durable background queue. If the process is down, recovery resumes only when the process runs again.
- Retries must be idempotent. Decided claims cannot be screened again; completed attempts must not receive duplicate matches or audit events.
- The UI must visibly distinguish PENDING, COMPLETED/MATCHES_FOUND, COMPLETED/NO_MATCH and FAILED.

3.3 Adjuster review and decision

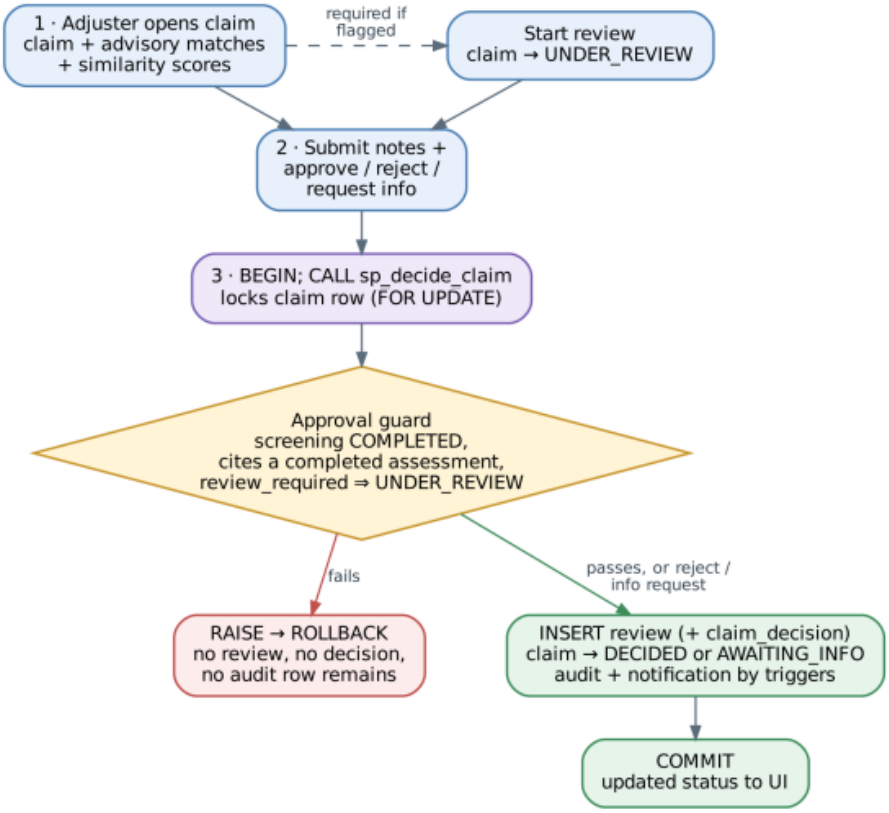


Figure 4. Decision flow. The guard and all writes run inside sp_decide_claim in one transaction.

Figure 6. Decision flow. The approval guard and writes are performed in one transaction.

9. An adjuster opens a claim and reviews screening state, advisory risk, rule version and matched reference cases.
10. If HIGH-risk review is required, the adjuster starts review and the claim enters UNDER_REVIEW.
11. The adjuster submits notes and one action: approve, reject or request information.
12. The decision procedure begins a transaction and locks the claim row to prevent concurrent decisions.
13. The guard verifies screening completion and mandatory review conditions for approval. Rejected approvals raise an error and roll back all writes.
14. On success, the procedure records review/decision and status change; database triggers write audit rows and create appropriate notification records in the same transaction.
15. After final decision, further screening, review, status and flag mutations are rejected.

4. Component contracts

4.1 Authentication and API handlers

- Resolve the authenticated session first, then enforce role and ownership, validate the request, call a service and map database errors to stable HTTP responses.
- Ownership checks must scope queries by the authenticated session user's ID; changing a claim ID in a URL must not reveal another user's claim.
- Public registration, if enabled, creates POLICYHOLDER accounts only. Admin and adjuster roles are assigned through authorized administrative setup.
- Every protected write uses a server-controlled transaction context for audit attribution.
- Do not return password hashes, raw secrets, stack traces or internal diagnostic details to the browser.

4.2 Embedding service

Endpoint	Contract

Endpoint	Contract
POST /embed	Input: texts array. Output: model_name, model_version, dimension=384 and normalized vector array. Reject empty/oversized input according to configured limits.
GET /health	Returns service health and whether the model is loaded; must not reveal secrets.
Network boundary	Local binding at 127.0.0.1:8001 by default; called by Next.js only. The service has no database credentials.

4.3 Logical API surface

Exact route paths must follow the checked-in repository. This table defines responsibilities, not a claim that every endpoint is already implemented.

Operation	Authorized actor	Expected behaviour
Session/authentication	All roles	Sign in/out and retrieve current session without exposing credential material.
List/create claim	Policyholder	List own claims; submit only against owned active policy.
Claim detail/status	Policyholder/Adjuster/Admin, field-limited	Return only data appropriate to the role; internal evidence is hidden from policyholders.
Review queue/search	Adjuster	Filter queue and inspect claim, screening, rule version and matched reference cases.
Screen/retry	System/Adjuster	Run screening or authorized bounded retry and persist attempt outcome.
Start review/decision	Adjuster	Start review; approve/reject/request information through guarded transaction.
Information response	Policyholder	Append response only for own claim in AWAITING_INFO.
Notifications	Authenticated user	List/read/dismiss only own notifications.
User/policy management	Admin	Manage users/roles and issue policies with validation and audit.
Reference/risk-rule management	Admin	Add/deactivate synthetic reference cases and create rule versions.
Audit/reporting	Admin; selected reports for adjusters	Read-only audit and role-appropriate operational/analytical reporting.

5. Database design and migration plan

The existing v3 SQL schema is the candidate migration 001 baseline. It must be inspected and run against a clean disposable database before adoption. Subsequent migrations should add only confirmed gaps, not duplicate existing objects.

5.1 Core entities

Entity group	Purpose and integrity rules
app_user / role	Identity, role and account state; password hashes only.
policy	Coverage dates, status and policyholder; valid coverage and incident-date checks.
claim	Original submission and lifecycle status; positive amount, valid policy relationship, immutable submitted content.
screening_attempt / screening_assessment	Attempt number, state, model identity, advisory risk and risk-rule version; bounded retry and explicit failure.
reference_fraud_case / embedding	Synthetic confirmed reference case; active/confirmed and compatible model required for retrieval.
screening_match	Ranked assessment-to-reference relationship with similarity score and rank integrity.
risk_rule	Versioned weights/thresholds; referenced rules are immutable and historical assessments preserve the version.

Entity group	Purpose and integrity rules
review / claim_decision	Review notes, information requests and final outcome; one final decision and approval guard.
claim_info_response	Append-only response history; responder must own claim and claim must be AWAITING_INFO.
notification	User-owned notification; dismissal cannot delete claim/decision/audit history.
audit_event	Actor/action/target/time/context; trigger-written transactionally and protected from ordinary mutation.

5.2 Migration sequencing

Migration	Scope (apply only after schema gap analysis)
001 — existing v3 baseline	Adopt the current verified schema as-is only after clean-database execution and test confirmation.
002 — screening/risk	Add versioned risk_rule, screening procedure and failure procedure, assessment fields, model-compatibility checks, retry guards and HIGH-risk trigger where absent.
003 — workflow/audit	Add missing policy issuance, information response, notification behaviour and audit triggers/guards where absent.
004 — search/reporting/recovery	Add full-text search vector/GIN index, missing report views, notification indexes and retry/idempotency support where absent.

Migration numbers above are a proposed grouping, not a command to create duplicate tables or triggers. Inspect current migrations first; update names and numbering to match the repository.

5.3 Database objects

Object	Role
Constraints and foreign keys	Enforce required values, positive amounts, date/ownership relationships and lifecycle integrity.
sp_record_screening (or verified equivalent)	Persist screening attempt, matches, risk assessment and outcome atomically.
sp_record_screening_failure (or equivalent)	Persist explicit FAILED outcome safely, even when screening cannot complete.
sp_decide_claim (or equivalent)	Lock claim, validate approval guard, record review/decision/status atomically.
Triggers	Write audit events, set HIGH-risk review flag, enforce immutable/final states and information-response transitions.
B-tree indexes	Support foreign keys, uniqueness, common filters and ordered queue queries.
GIN full-text index	Support ranked keyword search on claim descriptions.
HNSW vector index	Support approximate cosine nearest-neighbour retrieval.
Views	Provide operational summaries and review queue; add analytics views only when required.

6. Risk scoring and semantic retrieval

- Model: sentence-transformers/all-MiniLM-L6-v2; embedding dimension 384; vectors L2-normalized.
- Similarity metric: cosine similarity/distance. Retrieval is limited to active, confirmed reference cases with the same registered model identity.
- Default top-k is 5. The value must be compatible with the configured HNSW search settings and documented in rule/config data.
- Risk scoring runs in PostgreSQL from a versioned risk_rule. The exact formula and thresholds must match the checked-in migration/seed data and be tested; this document intentionally does not invent numeric cut-offs.
- Each assessment persists the rule version and model identity so the decision evidence can be reproduced.
- Similarity and risk are advisory signals only. Synthetic evaluation should record precision/recall for the HIGH flag and document class balance and limitations.

7. Security, integrity and reliability

Concern	Control
Authentication	Password hashing, secure session handling, no credential logging.
Authorization	Server-side role and ownership checks on every protected operation.
SQL injection	Parameterized SQL for every dynamic value.
Audit attribution	Trusted actor context set inside server transaction; triggers write audit rows in the same transaction.
Partial decision writes	One guarded database transaction for review/decision/status/audit/notification; failure rolls back all related writes.
Screening failure	Persist FAILED, block approval, show explicit failure and permit bounded retry.
Model mismatch	Store model identity/version; prohibit comparing vectors from different models.
History tampering	Append-only review/decision/audit and response history; deactivation instead of destructive removal for core records.
Secrets and synthetic data	Ignored local env file, committed env template, synthetic-only fixtures and screenshots.
Concurrency	Row locks/unique constraints for attempt numbers and one-decision-per-claim; test duplicate retries and concurrent decision attempts.

8. Reporting and required DBMS features

Report/query	SQL techniques demonstrated
Claims by status, screening state and flag	Views, GROUP BY, COUNT, ORDER BY.
Monthly risk distribution	JOIN, latest-assessment selection, date_trunc, GROUP BY.
Reference patterns matched repeatedly	JOIN, GROUP BY, HAVING, AVG.
Adjuster workload and approval rate	JOIN, conditional aggregates/FILTER, HAVING.
Claims above policyholder average	Correlated subquery and AVG.
Full-text claim search	PostgreSQL full-text vector, GIN index and relevance ranking.
Similarity retrieval	pgvector cosine distance, HNSW and top-k ordering.

9. Requirement traceability

Requirement IDs	Architecture/database mechanism	Verification
FR-01–FR-04; NFR-01	Authentication, server RBAC/ownership, claim API and constraints.	AC-01–AC-03; negative authorization tests.
FR-05–FR-07; FR-14; FR-22	Embedding service, screening orchestrator, pgvector retrieval, procedure, explicit states and bounded retry.	AC-04, AC-05, AC-10, AC-11, AC-15; failure/retry tests.
FR-08–FR-10; FR-13; BR-08–BR-11	Decision transaction, approval guard, append-only history and audit triggers.	AC-06, AC-07; invalid transition and rollback tests.
FR-11; FR-17; BR-02, BR-04	Admin module, policy/reference constraints and audit.	AC-02, AC-11.
FR-18–FR-19; BR-10, BR-14	Append-only responses and user-owned dismissible notifications.	AC-12, AC-13.
FR-12; FR-20–FR-21	Views, aggregates, GIN full-text index and report queries.	AC-08, AC-14.
NFR-02–NFR-10	Integrity, synthetic fixtures, setup docs, explicit failure states and evaluation records.	AC-05, AC-09, AC-15 and measured performance results.

10. Team responsibilities

Member	SRN	Proposed work split
Hrushikesh Karnam	PES1UG24CS668	Schema/data integrity review, claim workflow and screening integration, test coverage and documentation coordination.
Nishant Hegde	PES1UG24CS681	Architecture/API/service implementation, database procedure/trigger integration, review workflow and test coordination.

This split is a planning aid. Git history and actual contributions should be represented accurately; it must not be treated as proof of work already completed.

11. Implementation roadmap

16. Inspect current repository, schema v3, test suite, ER diagram and relational schema. Record which SRS requirements are already covered.
17. Run migrations on a clean disposable database and resolve any duplicate or missing objects before implementation.
18. Complete risk-rule versioning, screening success/failure persistence, audit triggers, approval guards, policy issuance, information responses, notifications and retry support.
19. Implement server-side session, role and ownership helpers; claim, screening, review and admin services.
20. Implement and verify Python embedding service, model identity checks, vector normalization and top-k retrieval.
21. Build role-specific UI, full-text search and reports after core lifecycle and security tests pass.
22. Run clean setup and negative tests for invalid transitions, unauthorized reads, service failure, concurrent retries, concurrent decisions and transaction rollback.
23. Update ER diagram, relational schema, data dictionary, README and screenshots to match verified code. Mark any incomplete Must requirement clearly.

12. Verification checklist and open decisions

- Confirm all table, procedure, trigger, view and index names against the current checked-in SQL.
- Confirm existing v3 object counts only by querying the schema; do not rely on old document counts.
- Verify all required audit events are generated by triggers and actor context cannot be spoofed by browser input.
- Test approval guard for PENDING/FAILED screening and unresolved HIGH-risk review.
- Test one-decision-per-claim, decided-claim immutability, idempotent retries and rollback behaviour.
- Verify policy date constraints, information-response ownership/state, notification dismissal and full-text search.
- Record actual test results, dataset composition and timing; do not state unverified pass counts or accuracy.

13. Final design position

ClaimGuard is designed as a small, locally runnable and auditable system. Next.js owns authenticated workflow and orchestration; Python owns embedding inference; PostgreSQL owns durable state and critical business invariants. The system surfaces similar confirmed synthetic patterns and a versioned advisory risk indicator. It does not automate a fraud determination or final claim decision.